



Ciência da Computação

Estrutura de Dados

Ponteiro

Aldo Moura
aldo.moura@souunit.com.br

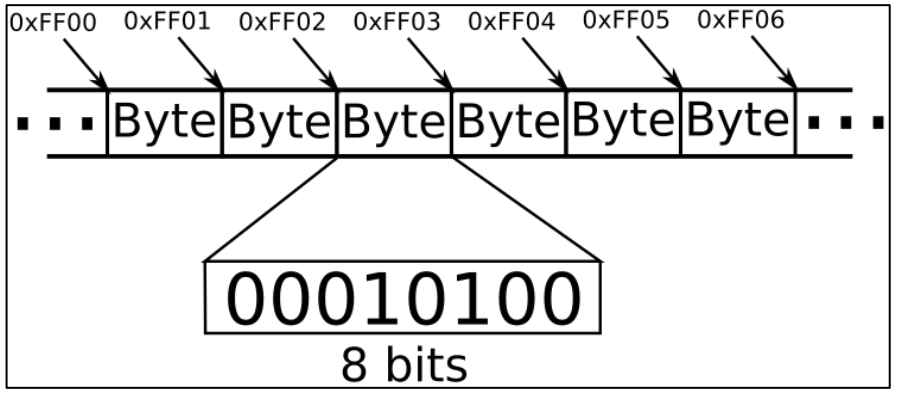
Objetivo e Roteiro



Compreender e aplicar o uso de ponteiro para resolver problemas computacionais.

Operador & (Endereço)

- Durante a execução de um programa suas variáveis ficam armazenadas na memória RAM do computador.
- Cada posição da memória guarda 1 byte.
- A posição da memória é denominada de endereço, normalmente representado em hexadecimal.



O operador & obtém o endereço de uma variável

```
double num1;
printf("%p", &num1);  /* %p especificador que mostra o endereço
                          de memória em hexadecimal */
```

- Ele só se aplica a objetos na memória: variáveis, vetores.

Operador sizeof

Cada variável ocupa um determinado número de bytes consecutivos na memória, por exemplo:

- char = 1 byte
- int = 4 bytes
- float = 4 bytes
- double = 8 bytes

O operador sizeof() fornece o número de bytes de uma variável.

```
double num1;
printf("%i", sizeof(num1));
```

Ponteiro (Apontador)

Tipo de variável que armazena o endereço de memória de outra variável.

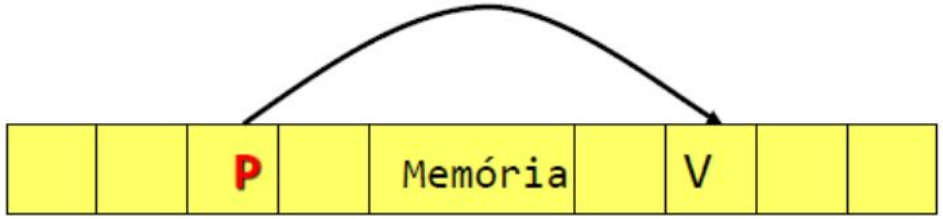
```
int  *p1;     // p1 é um ponteiro para inteiro
char *p2;     // p2 é um ponteiro para char

double num1 = 14.3;
double *p1;
p1 = &num1;
printf("Endereço da variável num1: %p", p1);
```

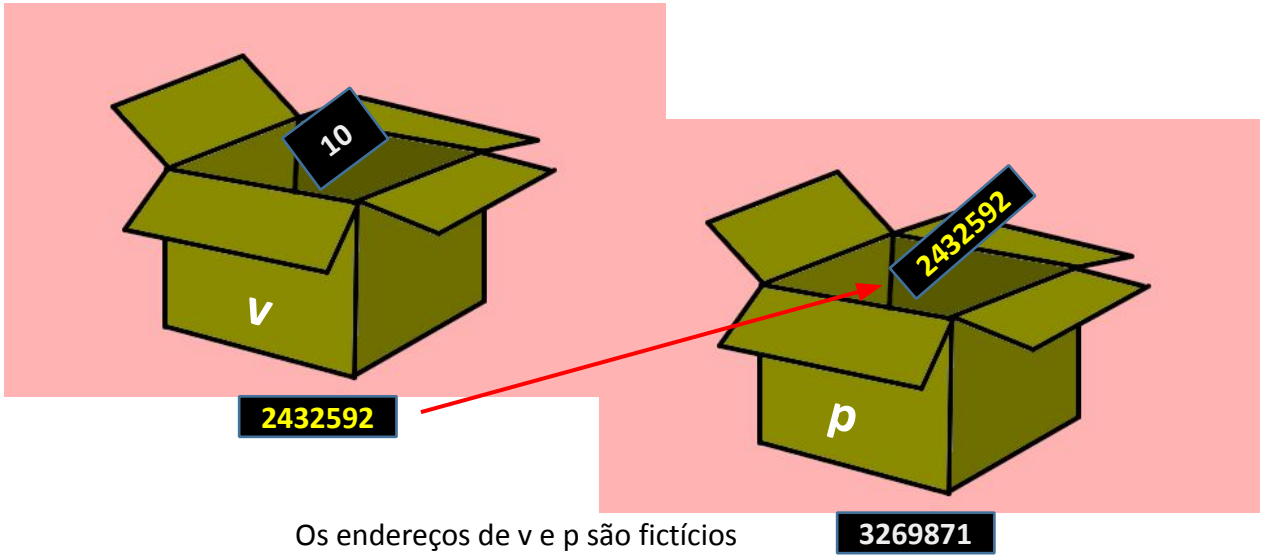
Operador * (Indireção)

Acessa um valor indiretamente, ou seja, manipula o valor no endereço para o qual o operando aponta.

```
int v;
int *p;
p = &v;   // p aponta para v
*p = 10; /* o endereço apontado
            por *p recebe 10 */
```



P armazena o endereço da variável V



Indireção: ato de referenciar algo de forma não direta, utilizando intermediários, referências ou nomes em vez do valor ou objeto real. Esse operador também é conhecido com desreferenciação ou operador de acesso direto a memória.

Vantagens do Uso de Ponteiro

- Acesso direto à memória, essencial em programação de baixo nível (drivers, sistemas embarcados).
- Alocação dinâmica de memória para estruturas que mudam de tamanho (listas, árvores, grafos).
- Passagem de parâmetros por endereço (referência), economizando memória ao evitar duplicidade de vetores/objetos grandes.
- Permite que funções alterem variáveis externas e retornem múltiplos valores (passagem por referência).
- Facilita a manipulação de arrays e strings (nomes funcionam como ponteiro para o 1º elemento).

Ponteiro

Considere o código a seguir:

```
int x, y;
x=5;
y=x;
x++;
```

Como atualizar o valor de y sempre que x for alterado, para que sempre y seja igual a x?

Solução 1: Sempre que uma linha alterar o valor de x, adicionar uma linha com o código y=x

```
x++;
y=x;
...
x = x * 2;
y=x;
```

Ponteiro

Considere o código a seguir:

```
int x, y;
x=5;
y=x;
x++;
```

Como atualizar o valor de y sempre que x for alterado, para que sempre y seja igual a x?

Solução 2: Ponteiro

```
int x, *y;
x=5;
y=&x;
x++;
```

Assim, y mantém o mesmo valor de x.

Passando Endereços Para Uma Função

Chamada por valor

```c
#include <stdio.h>

void altera(int x, int y)
{
    x = 3;
    y = 5;
}


int main()
{
    int x, y;
    x = 0;
    y = 0;
    altera(x, y);
    printf("1º numero: %d\n", x);
    printf("2º numero: %d\n", y);
    return 0;
}
```

As atribuições nas variávei x e y, que compõem a lista de parâmetros da função altera, não afetam os valores armazenados nas variáveis x e y declaradas na função main.

Chamada por referência

```c
#include <stdio.h>

void altera(int *x, int *y)
{
    *x = 3;
    *y = 5;
}


int main()
{
    int x, y;
    x = 0;
    y = 0;
    altera(&x, &y);
    printf("1º numero: %d\n", x);
    printf("2º numero: %d\n", y);
    return 0;
}
```

Os parâmetros *x e *y são ponteiros capazes de armazenar endereço de variáveis do tipo inteiro

As atribuições nas variáveis apontadas pelos ponteiros *x e *y, que compõem a lista de parâmetros da função altera, afetam nas variáveis x e y declaradas na função main.

Passando Endereços Para Uma Função

Como uma função chamada pode alterar as variáveis da função quem a chamou?

1) Na declaração da função chamada, os parâmetros que precisam ser alterados devem ser declarados como ponteiros

2) Na chamada da função, os argumentos que precisam ser alterados devem ser passados como endereços

Passando Endereços Para Uma Função

```c
#include <stdio.h>

int verifica(int n1, int n2, int *soma, int *produto)
{
    printf("N1: %d\n\n", n1);
    printf("N2: %d\n\n", n2);
    *soma = (n1 + n2);
    *produto = (n1 * n2);
    if (n1==n2)
    {
       return 0;
    }
    else if (n1>n2)
    {
       return 1;
    }
    else
    {
       return 2;
    }
}
```

```c
int main()
{
    int n1 = 10, n2 = 10, s = 0, p = 0;
    printf("Valor de Retorno: %i\n\n", verifica(n1, n2, &s, &p));
    printf("Soma = %i\n\n", s);
    printf("Produto = %i\n\n", p);
    return 0;
}
```

Operações com Ponteiros

A operação de incremento ou decremento em ponteiro muda o endereço que o ponteiro aponta em sizeof(tipo_variável) e não em unidade.

```
int x=3;      // 805616 suposto endereço de x
int y=6;      // 805620 suposto endereço de y
int *px=&x;   // 805624 suposto endereço de px
int *py=&y;   // 805632 suposto endereço de py

printf("px = %d, *px = %d, &px = %d\n", px, *px, &px);
printf("py = %d, *py = %d, &py = %d\n", py, *py, &py);
px++;
printf("px = %d, *px = %d, &px = %d\n", px, *px, &px);
```

```
px = 805616, *px = 3, &px = 805624
py = 805620, *py = 6, &py = 805632
px = 805620, *px = 6, &px = 805624
```

O operador `sizeof()` fornece o número de bytes de uma variável.

```
double num1;
printf("%ld", sizeof(num1));
```

O incremento de 1 em um ponteiro incrementa seu valor em 1 sizeof do tipo apontado.

Exemplo: Se px aponta para um int com o valor 805616, após a instrução px++ o valor de px será 805620 e não 805617.

Obs.: Isso depende de compilador para compilador, dependendo do número de bytes adotado para o referido tipo

Operações com Ponteiros

```
int x=3;          // 168320 suposto endereço de x
int y=6;          // 168324 suposto endereço de y
int *px=&x;       // 168328 suposto endereço de px
int *py=&y;       // 168336 suposto endereço de py

printf("px = %d, *px = %d, &px = %d\n", px, *px, &px);
printf("py = %d, *py = %d, &py = %d\n", py, *py, &py);
py = px + 3;
printf("py = %d, *py = %d, &py = %d\n", py, *py, &py);
```

```
px = 168320   *px = 3      &px = 168328
py = 168324   *py = 6      &py = 168336
py = 168332   *py = 32766  &py = 168336
```

Operações com Ponteiros

C tem um relacionamento muito forte entre ponteiros e vetores.

O compilador transforma todo vetor e matriz em ponteiros, pois a maioria dos computadores é capaz de manipular ponteiros e não vetores.

Qualquer operação que possa ser feita com índices de um vetor pode ser feita com ponteiros.

O nome de um vetor é um endereço, ou seja, um ponteiro.

Versão sem ponteiro

```
int numeros[] = {2, 4, 6};

for (int i=0; i<3; i++) {
     printf("%d\n", numeros[i]);
}
```

Versão com ponteiro

```
int numeros[] = {2, 4, 6};

for (int i=0; i<3; i++) {
     printf("%d\n", *(numeros + i));
}
```

Deslocamento

Endereço inicial do vetor

Operações com Ponteiros

```
Versão com ponteiro

int numeros[] = {2, 4, 6};

for (int i=0; i<3; i++) {
    printf("%d\n", *(numeros + i));
}
```

variável	Endereço de Memória
numero[0]	2022700
numero[1]	2022704
numero[2]	2022708

```
int numeros[] = {2, 4, 6};

for (int i=0; i<3; i++) {
    printf("%d\n", (numeros + i));
}
```

```
2
4
6
```

```
2022700
2022704
2022708
```

```
printf("%d", sizeof(numeros));
```

Qual valor será exibido?

Aplicação Usando Ponteiro

Calcular os termos de uma PA (Progressão aritmética)

Escreva um programa para receber o primeiro termo e a razão de uma PA. Calcular os dez primeiros termos da PA.

```
int pa[10], razao;
int *ptr;

printf("Insira o termo inicial da PA: ");
scanf("%d", &pa[0]);
printf("Insira razao da PA: ");
scanf("%d", &razao);

ptr = pa; // aponta para o 1º elemento do vetor (primeiro termo)
while( ptr != &pa[9])
{
    *(ptr+1) =  *ptr + razao; // próximo termo = termo atual + razão
    ptr++;
}

for(ptr = pa ; ptr <= &pa[9] ; ptr++) {
     printf("%d\n", *ptr);
}
```

Comparação de Ponteiro

Testes relacionais são aceitos entre ponteiros.

```
int x=3, y=6;
int *px=&x, *py=&y;

printf("px: %u\d\n", px);
printf("py: %u\d\n\n", py);

if (px < py){ // resulta em 1 (4 bytes de diferença)
    printf("py-px = %i\n", py-px);
    printf("X está armazenada um endereço abaixo de Y\n\n");
} else{
    printf("px-py = %i\n", px-py);
    printf("X está armazenada um endereço acima de Y\n\n");
}
```

A diferença entre dois ponteiros será dada na unidade do tipo de dado apontado.

Ponteiro para Estrutura

Acessando os membros através do ponteiro:

Normalmente, os membros de uma estrutura são acessados usando seu nome seguido do operador ponto. Em se tratando de ponteiro para struct, a codificação **ptrl.regnum** não é aceita.

Poderá ser usada umas das opções abaixo:

ptrl->regnum

(*ptrl).regum

Ponteiro para Estrutura

```
#include <stdio.h>

typedef struct livro
{
     char titulo[21];
     char autor[21];
     int regnum;
     double preco;
} tlivro;

int main ()
{
   tlivro lista[2] = { {"C", "Carlos", 102, 70.5}, {"C++", "Alexandre", 321, 63.25} };

   tlivro *ptrl; // ponteiro para estrutura
   ptrl = &lista[0];

   printf("Ponteiro 1   : %X2: %X\n", ptrl, ptrl+1);
   printf("Endereço 1   : %X2: %X\n\n", &lista[0], &lista[1]);
   printf("ptrl->preco  : R$%.2f\n", ptrl->preco);
   printf("(*ptrl).preco: R$%.2f\n\n", (*ptrl).preco);

   printf("ptrl->titulo: %-22s prtl->autor: %-35s\n", ptrl->titulo, ptrl->autor);
   ptrl++; // aponta para a próxima estrutura
   printf("ptrl->titulo: %-22s prtl->autor: %-35s\n", ptrl->titulo, ptrl->autor);
   return 0;
}
```

```
Ponteiro 1   : FC5F05E0        2: FC5F0618
Endereço 1   : FC5F05E0        2: FC5F0618

ptrl->preco  : R$70.50
(*ptrl).preco: R$70.50

ptrl->titulo: C                      prtl->autor: Carlos
ptrl->titulo: C++                    prtl->autor: Alexandre
```